

Azure Service Bus – Sending Messages in a Batch

1. What is Batch Message Sending?

Azure Service Bus allows us to send **multiple messages together as a batch** instead of sending each message individually.

Normally, if we have four messages, we could send them one by one:

```
```text id="a5x0qh" Send Message 1 Send Message 2 Send Message 3 Send Message 4
```

With batching, we create one message batch and add multiple messages to it:

```
```text id="x8f7hc"
Message 1
Message 2
Message 3
Message 4
    ↓
Message Batch
    ↓
Send Batch
```

This can be useful when an application has multiple messages ready to be sent at the same time.

2. Real-Time Scenario

Imagine we have a **scheduled service** that runs every hour.

For example:

```
```text id="y4f9x8" Scheduler ↓ Runs every 1 hour ↓ Collects data ↓ Creates messages ↓ Creates a batch
↓ Sends batch to Azure Service Bus
```

Suppose the scheduler collects four records:

```
```text id="4d8t0f"
Record 1
Record 2
```

Record 3
Record 4

Instead of sending them individually, we can create a batch and send them together.

3. Why Use Message Batching?

Batching is useful when we have multiple messages that need to be sent together.

Benefits include:

- Reduces the number of send operations.
- Can improve sending efficiency.
- Useful for applications processing messages in groups.
- Useful for scheduled or bulk-processing scenarios.

Simple Example

Without batching:

```text id="9ps5tv" Application ↓ Send Message 1 ↓ Send Message 2 ↓ Send Message 3 ↓ Send Message 4

With batching:

```
```text id="qf83yi"
Application
  ↓
Create Batch
  ↓
Message 1
Message 2
Message 3
Message 4
  ↓
Send Batch
```

4. Create Service Bus Client

First, create the Service Bus client:

```
```csharp id="d9yn3p" var client = new ServiceBusClient(connectionString);
```

The client is used to communicate with Azure Service Bus.

---

#### # 5. Create Service Bus Sender

Create a sender for the queue or topic:

```
```csharp id="n6j5o0"
var sender =
    client.CreateSender(queueName);
```

If we are sending to a topic:

```
```csharp id="6c9t0x" var sender = client.CreateSender(topicName);
```

The sender is responsible for sending messages.

---

#### # 6. Create a Message Batch

Instead of creating and sending messages individually, create a message batch:

```
```csharp id="e2y9rq"
using ServiceBusMessageBatch messageBatch =
    await sender.CreateMessageBatchAsync();
```

The important method is:

```
```text id="c2xj1w" CreateMessageBatchAsync()
```

This creates a batch that can contain multiple Service Bus messages.

---

#### # 7. Create Service Bus Messages

Suppose we have four messages:

```
```csharp id="9m1qzf"
```

```
var message1 =  
    new ServiceBusMessage("Message 1");  
  
var message2 =  
    new ServiceBusMessage("Message 2");  
  
var message3 =  
    new ServiceBusMessage("Message 3");  
  
var message4 =  
    new ServiceBusMessage("Message 4");
```

Now we have:

```
```text id="x8x3a9" Message 1 Message 2 Message 3 Message 4
```

```

8. Add Messages to the Batch

Add each message to the batch:

```csharp id="5y7nq1"  
messageBatch.TryAddMessage(message1);  
messageBatch.TryAddMessage(message2);  
messageBatch.TryAddMessage(message3);  
messageBatch.TryAddMessage(message4);
```

Now the batch contains:

```
```text id="w8p5d0" Message Batch
```

---

Message 1 Message 2 Message 3 Message 4

```

9. Important: TryAddMessage()

Notice that we use:

```csharp id="9q5p8e"  
TryAddMessage()
```

instead of directly adding the message.

Why?

Because a Service Bus message batch has a **maximum size**.

If adding a message would make the batch exceed the allowed size, `TryAddMessage()` returns `false`.

Example:

```
```csharp id="6o4zq7" if (!messageBatch.TryAddMessage(message)) { Console.WriteLine( "Message cannot fit in the batch."); }
```

So we should always check the return value when building batches dynamically.

---

# 10. Send the Message Batch

For a single message, we use:

```
```csharp id="2w3y8x"
await sender.SendMessageAsync(message);
```

For multiple messages in a batch, we use:

```
```csharp id="4g2w8j" await sender.SendMessagesAsync(messageBatch);
```

### Important Difference

```
```text id="r9l1hv"
```

Single Message:

`SendMessageAsync()`

Message Batch:

`SendMessagesAsync()`

Remember this for interviews.

11. Complete Example

A simplified example:

```
```csharp id="f4q6d2" var client = new ServiceBusClient(connectionString);

var sender = client.CreateSender(queueName);

using ServiceBusMessageBatch messageBatch = await sender.CreateMessageBatchAsync();

var message1 = new ServiceBusMessage("Message 1");

var message2 = new ServiceBusMessage("Message 2");

var message3 = new ServiceBusMessage("Message 3");

var message4 = new ServiceBusMessage("Message 4");

messageBatch.TryAddMessage(message1); messageBatch.TryAddMessage(message2);
messageBatch.TryAddMessage(message3); messageBatch.TryAddMessage(message4);

await sender.SendMessagesAsync(messageBatch);
```

---

# 12. End-to-End Flow

```text id="d3r9w8"

Application / Scheduler

↓

Create Service Bus Client

↓

Create Service Bus Sender

↓

Create Message Batch

↓

Create Message 1

Create Message 2

Create Message 3

Create Message 4

↓

Add messages to batch

↓

SendMessagesAsync()

↓

Azure Service Bus Queue/Topic

13. Testing in Azure Portal

Before sending the messages, suppose the queue contains:

```
```text id="x5p1v7" Message Count = 0
```

After executing the program:

```
```text id="k9y3c2"
Message Count = 4
```

This confirms that four messages have been sent.

14. Verify Using Service Bus Explorer

Open:

```
```text id="5b0g8p" Azure Portal ↓ Service Bus ↓ Queue ↓ Service Bus Explorer
```

Select **\*\*Peek / Pick from Start\*\***.

You should see:

```
```text id="r4c1g7"
Message 1
Message 2
Message 3
Message 4
```

For example:

```
```text id="9j8v4d" Message 1
```

and:

```
```text id="p6z3k1"
Message 4
```

This confirms that all four messages were sent successfully.

15. Important Concept – Batch vs Single Message

Sending a Single Message

```
```csharp id="s4b7kx" await sender.SendMessageAsync(message);
```

Example:

```
```text id="o9s3q5"
Message 1 → Send
Message 2 → Send
Message 3 → Send
Message 4 → Send
```

This requires multiple send operations.

Sending a Batch

```
```csharp id="t7w2n4" await sender.SendMessagesAsync(messageBatch);
```

Example:

```
```text id="3s6k2d"
Message 1
Message 2
Message 3
Message 4
    ↓
    Batch
    ↓
Send once
```

16. Real-Time Example – Scheduled Job

Suppose an application runs a scheduled job every hour.

At 10:00 AM:

```text id="x6r8v0" Scheduler starts ↓ Collects 100 records ↓ Creates Service Bus messages ↓ Creates batches ↓ Sends batches to Service Bus

For example:

```
```text id="q8f5m2"
100 records
  ↓
Message 1
Message 2
...
Message 100
  ↓
Create batches based on size
  ↓
Send batches
```

This is much more suitable than making a separate send operation for every message when you already have a group of messages ready to send.

17. Important: Batch Size

A batch does **not** mean we can put an unlimited number of messages into it.

The batch has a maximum size.

Therefore:

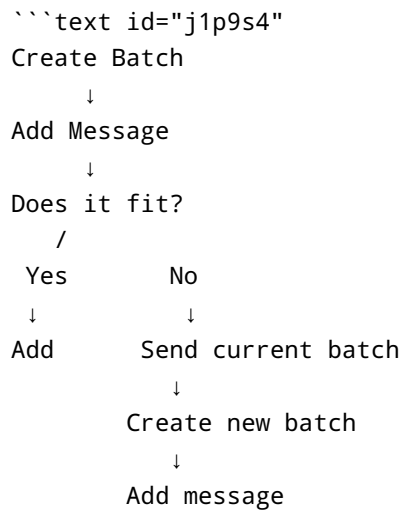
```
```csharp id="u3g5s8" if (!messageBatch.TryAddMessage(message)) { // Batch is full }
```

If the message cannot fit:

1. Send the current batch.
2. Create a new batch.
3. Add the message to the new batch.

4. Continue processing.

Conceptually:



## 18. Dynamic Batch Example

When processing many messages, we can use a pattern like:

```
```csharp id="r8t2m6" using var batch = await sender.CreateMessageBatchAsync();
```

```
foreach (var item in messages) { var message = new ServiceBusMessage(item);
```

```
    if (!batch.TryAddMessage(message))
    {
        await sender.SendMessagesAsync(batch);

        batch = await sender.CreateMessageBatchAsync();

        batch.TryAddMessage(message);
    }
```

```
}
```

```
await sender.SendMessagesAsync(batch);
```

The exact implementation may need adjustment because the batch object should be managed carefully when recreated, but the important concept is:

```
```text id="v4j7n2"
Add messages
 ↓
If batch is full
 ↓
Send batch
 ↓
Create new batch
 ↓
Continue
```

## 19. When Should We Use Batch Sending?

Batch sending is useful when:

### 1. Multiple messages are available

Example:

```
```text id="g7r1m8" 100 messages ready
```

Instead of sending each separately, create batches.

2. Scheduled Jobs

Example:

```
```text id="e8q2w5"
Job runs every hour
 ↓
Collect data
 ↓
Create messages
 ↓
Send batches
```

### 3. Bulk Processing

Example:

```
```text id="n4k8x1" Database records ↓ Convert to messages ↓ Send in batches
```

4. Improving Send Efficiency

Batching can reduce the number of individual send operations and improve efficiency when sending many messages.

20. When Should We NOT Think of Batching as Necessary?

If an application receives a single event and needs to send it immediately, batching may not provide much benefit.

Example:

```
```text id="q3f6m9"
User places order
 ↓
Create one message
 ↓
Send immediately
```

In this case:

```
```csharp id="b5n7x3" await sender.SendMessageAsync(message);
```

is appropriate.

21. Queue vs Topic

Message batching can be used when sending messages to:

Queue

```
```text id="w8q1c5"
Sender
 ↓
Queue
```

### Topic

```
```text id="m6r9t2" Sender ↓ Topic ↓ Subscriptions
```

The sending concept is the same:

```
```csharp id="b2k7s4"
await sender.SendMessagesAsync(messageBatch);
```

---

## 22. Important Interview Questions

### Q1. How do you send multiple messages to Azure Service Bus?

Create a `ServiceBusMessageBatch`, add messages using `TryAddMessage()`, and send the batch using `SendMessagesAsync()`.

---

### Q2. What is the difference between `SendMessageAsync` and `SendMessagesAsync`?

```text id="e7v3m9" `SendMessageAsync()` → Sends one message.

`SendMessagesAsync()` → Sends multiple messages or a message batch.

```
---

## Q3. How do you create a message batch?

```csharp id="f2k8p6"
var batch =
 await sender.CreateMessageBatchAsync();
```

---

### Q4. Why does `TryAddMessage` return a Boolean?

Because the message may not fit into the current batch due to the batch size limit.

```text id="a4x6z8" true → Message was added.

false → Message could not fit.

```
---
```

Q5. What should we do when TryAddMessage returns false?

Send the current batch, create a new batch, and then add the message to the new batch.

Q6. Can we send a batch to a queue?

Yes.

A batch can be sent to an Azure Service Bus queue.

Q7. Can we send a batch to a topic?

Yes.

A batch can also be sent to an Azure Service Bus topic.

Q8. Does batching mean all messages are processed as one message?

No.

A batch is a group of individual Service Bus messages sent together. The receiver still receives individual messages.

23. Important Points for Quick Revision

- **Batching** means sending multiple Service Bus messages together.
- Create a batch using:

```
```csharp id="g3x9p2"  
CreateMessageBatchAsync()
```

- Add messages using:

```
```csharp id="h6k1r4" TryAddMessage()
```

- Send the batch using:

```
```csharp id="j8m5t7"
SendMessagesAsync()
```

- `TryAddMessage()` returns `false` when the message cannot fit into the batch.
  - A batch has a maximum size.
  - If the batch becomes full, send it and create another batch.
  - Batching is useful for bulk processing and scheduled jobs.
  - Batching can reduce the number of individual send operations.
  - Batching can be used with both **queues and topics**.
  - A batch contains multiple individual messages; it does not turn them into one logical message.
- 

## 24. Easy Memory Trick

```text id="q2w5e8" Single Message ↓ SendMessageAsync()

Multiple Messages ↓ CreateMessageBatchAsync() ↓ TryAddMessage() ↓ SendMessagesAsync() ```

One-Line Interview Answer

Azure Service Bus message batching allows us to group multiple individual messages into a batch and send them together using `SendMessagesAsync()`, which is useful for bulk or scheduled message processing and can improve sending efficiency.